present a pretty aesthetic boot experience to the end user. Kernel Modesetting (KMS) and a new bootsplash program called Plymouth is utilised.

Before I start on Fedora 12's new features, there are certain technologies that I must list, in order to explain what's new in this release. They are:

- **Nash:** For all practical purposes, this is a minimalistic shell that fits into the *initrd* and executes init scripts that are present in the *initrd* image. Technically, however, it's a whole new story. Nash is a script interpreter that was specifically designed to interpret Linux RC scripts in *initrd* images. As such, it has a lot of built-in commands to this effect, like *mkrootdev, mkdevices* and *switchroot*. Its syntax is BASH compatible and the scripts are interchangeable as long as no Nash-specific built-in commands are used.
- **initrd:** I've been ranting about this a lot. Basically, when the kernel switches your processor to 32-bit protected mode, it loses all I/O support from the BIOS and must load a few I/O controller drivers before it can see the hard drives. So before the system goes 32-bit, a virtual disk is created in RAM, which holds certain files—mostly I/O drivers and a few initialisation scripts (Linux RC). These files are used to load in support for disk drives afterwards, and then switch to the root filesystem.
- **Modesetting:** This basically means setting the monitor's screen resolution and colour depth (i.e., mode). There are two ways in which modes can be set, either by the kernel or by programs in user space. They are called KMS and UMS, respectively.
- **RHGB (Red Hat Graphical Boot):** In Fedora releases, up to version 8, this is what provided the graphical boot experience by using a separate GUI program tacked on top of a separate instance of X11. This has been replaced by Plymouth.
- **mkinitrd:** This tool (basically a shell script) creates an *initrd* image. Without a managed framework or a set of guidelines on how this tool is supposed to work, almost every distro uses a separate implementation of the script and the script generally creates a monolithic *initrd* image. Dracut replaces this, and introduces a distro-independent modular architecture as a replacement.

Well, now that we are done with the introductions, let's try to understand these separate technologies that together power up Fedora.

## Dracut

"The current incarnation of *mkinitrd* sucks for a variety of reasons." When developers say that, it probably means doomsday is looming. So, rather than waiting for upstream to come up with something cool, Fedora developers took up the charge of creating an "*initramfs* infrastructure" that has "...as little as possible hard-coded into the *initramfs*." And it looks like they made it big.

The previous implementation of the *initramfs* did 'suck'. It was a completely custom system, with different images for different things—such as one for the Live CD, one for a running system and one for installation. It had a lot of code that was custom built only for Fedora and, that too, only for one particular type of the distribution. Moreover, an entirely different shell, called Nash, was used to support the init scripts. The new system does away with all these and produces a new, modular, event-driven, distro-independent and one-size-fits-all tool. Copied verbatim from Dracut's Trac wiki, this is what it is all about:

"Unlike existing *initramfs's*, this is an attempt at having as little as possible hard-coded into the *initramfs*. The *initramfs* has (basically) one purpose in life—getting the root FS mounted so that we can transition to the real root FS. This is all driven off of device availability. Therefore, instead of scripts hard-coded to do various things, we depend on udev to create device nodes for us, and then when we have the root FS's device node, we mount and carry on. This helps to keep the time required in the *initramfs* as little as possible, so that things like a 5-second boot aren't made impossible as a result of the very existence of an *initramfs*."

Basically, Dracut aims for (again, copied verbatim from the website):

- One *initrd* to rule them all, instead of one for the Live CD, one for the installer, and one for the distro.
- *Death to Nash:* The bits of Nash that don't currently exist in other tools, need factoring out and putting into *util-linux* or similar packages.
- *Same bits:* Use the same bits as the rest of the distro uses, instead of custom functionality. For example, there's no need for a custom udev, etc.
- *Diagnostics:* Include a shell for diagnostic/debugging purposes.
- *Probe at runtime instead of build time:* Include all drivers, not just the drivers necessary for the hardware being run. (For storage, the common ones will be built into the kernel anyway.)
- *The initrd will be usable on other distros:* The idea being that it can be included in the upstream kernel, and changed when kernel interfaces change, etc, to prevent breaking distros.

The Dracut architecture is event-driven and makes use of a Bash shell, eliminating Nash from *initrd's* picture. This has a three-fold advantage over using a minimal shell or a home-grown replacement.

One, bugs are easier to fix and adding features becomes less tedious, as it has to be done in only one place. Two, the danger of potentially different behaviour when running under two different shells, is eliminated. And three, because an existing popular technology is used, chances are that people already know how to work with it and don't need to learn anything new.

The end result is that Nash isn't really required any more, and it will be dropped in a future release. However, in Fedora 12, Nash is still used in a few places including Anaconda, the system installer.